

Assignment No. : 1

Course Title:

Data Structures

Course Code:

SE-200

Course Section:

C

Instructor:

Muhammad Sabeeh

Submitted by:

Muhammad Ali Abid

Roll Number:

25021598-010

Submitted to:

Muhammad Sabeeh

Date:

Table of Contents:

Question 1 – Array Operations	3
Java Code:.....	3
Sample Output:	8
Question 2 – Stack Operations	9
Java Code:.....	9
Sample Output:	12
Question 3 – Queue Operations.....	13
Java Code:.....	13
Sample Output:	17
Question 4 – Real World Examples	17
1. Array:	17
2. Stack:.....	18
3. Queue:	18
Question 5 — Advantages and Disadvantages.....	19
Array:	19
Stack:.....	19
Queue:	19

Question 1 -- Array Operations

Scenario:

A small library wants a simple program to manage the list of book IDs currently on a shelf. The shelf can hold a limited number of books at a time, and the librarian needs a menu-driven console program to manage this list.

Requirements:

An array-based program was implemented in Java, using a single fixed-size array (10 slots) to represent the shelf. The program supports the following operations through a console menu: Add value, Insert at index, Fill array, Delete last element, Delete by index, Display, Search value, Get value at index, Replace/Update value at index, Size, and Exit.

Java Code:

```
import java.util.*;

class Assignment_1_Program_1 {

    public static final String RESET    = "\u001B[0m";
    public static final String BOLD     = "\u001B[1m";
    public static final String CYAN     = "\u001B[36m";
    public static final String GREEN    = "\u001B[32m";
    public static final String YELLOW   = "\u001B[33m";
    public static final String PURPLE   = "\u001B[35m";
    public static final String RED       = "\u001B[31m";
    public static final String BLUE     = "\u001B[34m";
    public static final String WHITE    = "\u001B[37m";

    private static final Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        int[] books = new int[10];
        printBanner();
        loadingAnimation("Initializing Library Shelf Manager");

        boolean running = true;
        while (running) {
            displayMenu();
            int choice = readChoice();

            switch (choice) {
                case 1: {
                    printActionHeader("Add value");
                    AdditionOFBooks additionOFBooks = new AdditionOFBooks();
                    additionOFBooks.addBook(books);
                    break;
                }
                case 2: {
                    printActionHeader("Insert at index");
                    AdditionOFBooks additionOFBooks = new AdditionOFBooks();
                    additionOFBooks.indexInsertion(books);
                    break;
                }
                case 3: {
                    printActionHeader("Fill array");
                    AdditionOFBooks additionOFBooks = new AdditionOFBooks();
                    additionOFBooks.fillArray(books);
                    break;
                }
                case 4: {
                    printActionHeader("Delete last element");
                    DeletionBooks deletionBooks = new DeletionBooks();
                    deletionBooks.deleteLastElement(books);
                    break;
                }
                case 5: {
                    printActionHeader("Delete by index");
```



```

String border = "=".repeat(38);

System.out.println(CYAN + BOLD + "┌" + border + "┐" + RESET);
System.out.println(CYAN + BOLD + "│" + centerText(title, 38) + "│" + RESET);
System.out.println(CYAN + BOLD + "└" + border + "┘" + RESET);

printMenuItem(1, "Add value", GREEN);
printMenuItem(2, "Insert at index", GREEN);
printMenuItem(3, "Fill array", GREEN);
printMenuItem(4, "Delete last element", YELLOW);
printMenuItem(5, "Delete by index", YELLOW);
printMenuItem(6, "Display", BLUE);
printMenuItem(7, "Search value", BLUE);
printMenuItem(8, "Get value at index", PURPLE);
printMenuItem(9, "Replace/Update value at index", PURPLE);
printMenuItem(10, "Size", WHITE);
printMenuItem(11, "Exit", RED);

System.out.println(CYAN + BOLD + "└" + border + "┘" + RESET);
System.out.print(BOLD + YELLOW + "\n► Enter your choice: " + RESET);
}

private static void printMenuItem(int number, String label, String color) {
    String numStr = String.format("%2d", number);
    System.out.println(CYAN + "│" + RESET + color + BOLD + numStr + ". " + RESET
        + String.format("%-33s", label) + CYAN + "│" + RESET);
}

private static String centerText(String text, int width) {
    int padding = (width - text.length()) / 2;
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < padding; i++) sb.append(" ");
    sb.append(BOLD).append(text).append(RESET).append(CYAN).append(BOLD);
    while (sb.length() < width + (BOLD + RESET + CYAN + BOLD).length()) sb.append(" ");
    return sb.toString();
}

private static int readChoice() {
    int choice = -1;
    try {
        choice = Integer.parseInt(sc.nextLine().trim());
    } catch (NumberFormatException e) {
        choice = -1;
    }
    return choice;
}

private static void loadingAnimation(String message) {
    System.out.print(PURPLE + BOLD + message + RESET);
    for (int i = 0; i < 5; i++) {
        System.out.print(GREEN + "." + RESET);
        sleep(200);
    }
    System.out.println(GREEN + BOLD + " Done!\n" + RESET);
}

private static void printActionHeader(String action) {
    System.out.println(BOLD + GREEN + "\n► " + action + " selected...\n" + RESET);
}

private static void pauseForEffect() {
    System.out.print(WHITE + "\nPress ENTER to continue..." + RESET);
    sc.nextLine();
}

private static void exitAnimation() {
    System.out.println(YELLOW + BOLD + "\nClosing Library Shelf Manager");
    for (int i = 0; i < 3; i++) {
        System.out.print(YELLOW + "." + RESET);
        sleep(300);
    }
}

```

```

    }
    System.out.println(GREEN + BOLD + "\n✓ Goodbye!\n" + RESET);
}

private static void clearConsoleEffect() {
    System.out.print("\n".repeat(1));
}

private static void sleep(int ms) {
    try {
        Thread.sleep(ms);
    } catch (InterruptedException ignored) {
    }
}
}

class AdditionOFBooks {
    Scanner scanner = new Scanner(System.in);

    public void displayBook(int[] books) {
        if(books.length == 0) {
            System.out.println("Array is empty. No book IDs to display.");
        } else {
            System.out.println("Current book IDs in the array:");
            for(int i = 0; i < books.length; i++) {
                System.out.println("Index " + i + ": Book ID " + books[i]);
            }
        }
    }

    public void addBook(int[] books) {
        if(books != null && books.length > 0) {
            System.out.print("Enter the number of books to add (up to " + books.length + "): ");
            int count = scanner.nextInt();
            for(int i = 0; i < count && i < books.length; i++) {
                System.out.print("Enter book ID for book " + (i + 1) + ": ");
                int bookId = scanner.nextInt();
                books[i] = bookId;
            }
        }
    }

    public void indexInsertion(int[] books) {
        System.out.print("How many books do you want to insert? ");
        int count = scanner.nextInt();
        if(count <= books.length) {
            for(int i = 0; i < count; i++) {
                System.out.print("Enter the index where you want to insert the book ID: ");
                int index = scanner.nextInt();
                if(index >= 0 && index < books.length) {
                    System.out.print("Enter the book ID to insert: ");
                    int bookId = scanner.nextInt();
                    books[index] = bookId;
                } else {
                    System.out.println("Invalid index. Please enter a value between 0 and " + (books.length -
1));
                }
            }
        } else {
            System.out.println("You cannot insert more books than the array size.");
        }
    }

    public void fillArray(int[] books) {
        if(books != null && books.length > 0) {
            System.out.println("Filling the array with book IDs.");
            for(int i = 0; i < books.length; i++) {
                System.out.print("Enter book ID for book " + (i + 1) + ": ");
                books[i] = scanner.nextInt();
            }
        } else {
    }

```

```

        System.out.println("The array is empty or not initialized.");
    }
}

class DeletionBooks {
    Scanner scanner = new Scanner(System.in);

    public void deleteLastElement(int[] books) {
        int size = books.length;
        if(size > 0) {
            books[size - 1] = 0;
            System.out.println("Last book ID deleted.");
        } else {
            System.out.println("Array is empty. No book ID to delete.");
        }
    }

    public void deleteByIndex(int[] books) {
        if(books.length == 0){
            System.out.println("Array is empty. No book ID to delete.");
        } else {
            System.out.println("Array contents:");
            for(int i = 0; i < books.length; i++) {
                System.out.println("Index " + i + ": Book ID " + books[i]);
            }
            System.out.print("Enter the index of the book ID to delete (0 to " + (books.length - 1) + "): ");
            int index = scanner.nextInt();
            if(index >= 0 && index < books.length) {
                books[index] = 0;
                System.out.println("Book ID at index " + index + " deleted.");
            } else {
                System.out.println("Invalid index. Please enter a value between 0 and " + (books.length - 1));
            }
        }
    }
}

class UpdationBooks {
    Scanner scanner = new Scanner(System.in);

    public void searchValue(int[] books) {
        if(books.length == 0) {
            System.out.println("Array is empty. No book ID to search.");
        } else {
            System.out.print("Enter the book ID to search for: ");
            int bookId = scanner.nextInt();
            boolean found = false;
            for(int i = 0; i < books.length; i++) {
                if(books[i] == bookId) {
                    System.out.println("Book ID " + bookId + " found at index " + i);
                    found = true;
                    break;
                }
            }
            if(!found) {
                System.out.println("Book ID " + bookId + " not found.");
            }
        }
    }

    public void displayByIndex(int[] books) {
        if(books.length == 0) {
            System.out.println("Array is empty. No book ID to display.");
        } else {
            System.out.print("Enter the index of the book ID to display (0 to " + (books.length - 1) + "): ");
            int index = scanner.nextInt();
            if(index >= 0 && index < books.length) {
                System.out.println("Book ID at index " + index + ": " + books[index]);
            } else {
                System.out.println("Invalid index. Please enter a value between 0 and " + (books.length - 1));
            }
        }
    }
}

```

```

    }
}

public void updateValueAtindex(int[] books) {
    if(books.length == 0) {
        System.out.println("Array is empty. No book ID to update.");
    } else {
        System.out.println("Array Content:");
        for(int i = 0; i < books.length; i++) {
            System.out.println("Index " + i + ": Book ID " + books[i]);
        }
        System.out.print("Enter the index of the book ID to update (0 to " + (books.length - 1) + "): ");
        int index = scanner.nextInt();
        if(index >= 0 && index < books.length) {
            System.out.print("Enter the new book ID to replace the old one: ");
            int newBookId = scanner.nextInt();
            books[index] = newBookId;
            System.out.println("Book ID at index " + index + " updated to " + newBookId);
        } else {
            System.out.println("Invalid index. Please enter a value between 0 and " + (books.length - 1));
        }
    }
}
}
}

```

Sample Output:

```

LIBRARY SHELF MANAGER

Initializing Library Shelf Manager..... Done!

ARRAY MENU

1. Add value
2. Insert at index
3. Fill array
4. Delete last element
5. Delete by index
6. Display
7. Search value
8. Get value at index
9. Replace/Update value at index
10. Size
11. Exit

```

Console output of the Array (Library Shelf Manager) program -- Main menu.

Question 2 -- Stack Operations

Scenario

A pile of plates, each carrying a whole number, is stacked one on top of another. A new plate can only be added on top, and only the topmost plate can be removed. A program was built to simulate this Last-In-First-Out (LIFO) stack of integer values.

Requirements

A stack-based program was implemented in Java using a single array and a top variable (no built-in Stack class). The program supports: Push, Pop, Display, Size, isEmpty, isFull, and Exit, through a console menu.

Java Code:

```
import java.util.*;

public class Assignment_1_Program_2 {
    public static final String RESET = "\u001B[0m";
    public static final String BOLD = "\u001B[1m";
    public static final String CYAN = "\u001B[36m";
    public static final String GREEN = "\u001B[32m";
    public static final String YELLOW = "\u001B[33m";
    public static final String PURPLE = "\u001B[35m";
    public static final String RED = "\u001B[31m";
    public static final String BLUE = "\u001B[34m";
    public static final String WHITE = "\u001B[37m";

    private static final Scanner sc = new Scanner(System.in);

    public static void main(String[] args) {
        printBanner();
        loadingAnimation("Initializing Stack Manager");

        System.out.print(BOLD + YELLOW + "\nEnter the maximum capacity of the stack: " + RESET);
        int capacity = readIntInput();
        Plate stack = new Plate(capacity);
        boolean running = true;
        while (running) {
            displayMenu();
            int choice = readChoice();

            switch (choice) {
                case 1: {
                    printActionHeader("Push");
                    System.out.print(BOLD + YELLOW + "Enter the plate number to push: " + RESET);
                    int plateNumber = readIntInput();
                    stack.push(plateNumber);
                    break;
                }
                case 2: {
                    printActionHeader("Pop");
                    stack.pop();
                    break;
                }
                case 3: {
                    printActionHeader("Display");
                    stack.display();
                    break;
                }
                case 4: {
                    printActionHeader("Size");
                    int size = stack.size();
                    System.out.println(BOLD + YELLOW + "Current size of the stack: " + size + RESET);
                    break;
                }
                case 5: {
                    printActionHeader("isEmpty");
                }
            }
        }
    }
}
```



```

        printMenuItem(7, "Exit", RED);

        System.out.println(CYAN + BOLD + "┌" + border + "┐" + RESET);
        System.out.print(BOLD + YELLOW + "\n► Enter your choice: " + RESET);
    }

    private static void printMenuItem(int number, String label, String color) {
        String numStr = String.format("%2d", number);
        System.out.println(CYAN + "┌" + RESET + color + BOLD + numStr + ". " + RESET
            + String.format("%-29s", label) + CYAN + "┐" + RESET);
    }

    private static String centerText(String text, int width) {
        int padding = (width - text.length()) / 2;
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < padding; i++) sb.append(" ");
        sb.append(BOLD).append(text).append(RESET).append(CYAN).append(BOLD);
        while (sb.length() < width + (BOLD + RESET + CYAN + BOLD).length()) sb.append(" ");
        return sb.toString();
    }

    private static int readChoice() {
        return readIntInput();
    }

    private static int readIntInput() {
        while (true) {
            try {
                return Integer.parseInt(sc.nextLine().trim());
            } catch (NumberFormatException e) {
                System.out.print(RED + BOLD + "Invalid input. Please enter a valid number: " + RESET);
            }
        }
    }

    private static void loadingAnimation(String message) {
        System.out.print(PURPLE + BOLD + message + RESET);
        for (int i = 0; i < 5; i++) {
            System.out.print(GREEN + "." + RESET);
            sleep(200);
        }
        System.out.println(GREEN + BOLD + " Done!\n" + RESET);
    }

    private static void printActionHeader(String action) {
        System.out.println(BOLD + GREEN + "\n► " + action + " selected...\n" + RESET);
    }

    private static void pauseForEffect() {
        System.out.print(WHITE + "\nPress ENTER to continue..." + RESET);
        sc.nextLine();
    }

    private static void exitAnimation() {
        System.out.println(YELLOW + BOLD + "\nClosing Stack Manager");
        for (int i = 0; i < 3; i++) {
            System.out.print(YELLOW + "." + RESET);
            sleep(300);
        }
        System.out.println(GREEN + BOLD + "\n✓ Goodbye!\n" + RESET);
    }

    private static void clearConsoleEffect() {
        System.out.println("\n".repeat(1));
    }

    private static void sleep(int ms) {
        try {
            Thread.sleep(ms);
        } catch (InterruptedException ignored) {

```

```

    }
}

class Plate {

    private int[] plates;
    private int top;
    private int capacity;

    public Plate(int capacity) {
        this.capacity = capacity;
        plates = new int[capacity];
        top = -1;
    }

    public void push(int plate) {
        if(isFull()) {
            System.out.println("Stack is full. Cannot push plate " + plate);
        } else {
            plates[++top] = plate;
            System.out.println("Plate " + plate + " pushed to stack.");
        }
    }

    public int pop() {
        if(isEmpty()) {
            System.out.println("Stack is empty. Cannot pop.");
            return -1;
        } else {
            int popped = plates[top--];
            System.out.println("Plate " + popped + " popped from stack.");
            return popped;
        }
    }

    public void display() {
        if(isEmpty()) {
            System.out.println("Stack is empty.");
        } else {
            System.out.println("Current stack (top to bottom):");
            for(int i = top; i >= 0; i--) {
                System.out.println(plates[i]);
            }
        }
    }

    public boolean isFull() {
        return top == capacity - 1;
    }

    public boolean isEmpty() {
        return top < 0;
    }

    public int size() {
        return top + 1;
    }
}

```

Sample Output:


```

        for (int i = 0; i < banner.length; i++) {
            System.out.println(BOLD + colors[i] + banner[i] + RESET);
            sleep(80);
        }
        System.out.println(BOLD + YELLOW + "          F I F O   Q U E U E   M A N A G E R" + RESET);
        System.out.println(GREEN + "          " + "-" .repeat(40) + RESET);
        sleep(300);
    }

    private static void displayMenu() {
        clearConsoleEffect();
        String title = "QUEUE MENU";
        String border = "=" .repeat(34);

        System.out.println(CYAN + BOLD + "┌" + border + "┐" + RESET);
        System.out.println(CYAN + BOLD + "│" + centerText(title, 34) + "│" + RESET);
        System.out.println(CYAN + BOLD + "└" + border + "┘" + RESET);

        printMenuItem(1, "Enqueue", GREEN);
        printMenuItem(2, "Dequeue", GREEN);
        printMenuItem(3, "Display", BLUE);
        printMenuItem(4, "Size", BLUE);
        printMenuItem(5, "isEmpty", PURPLE);
        printMenuItem(6, "isFull", PURPLE);
        printMenuItem(7, "Exit", RED);

        System.out.println(CYAN + BOLD + "└" + border + "┘" + RESET);
        System.out.print(BOLD + YELLOW + "\n➤ Enter your choice: " + RESET);
    }

    private static void printMenuItem(int number, String label, String color) {
        String numStr = String.format("%2d", number);
        System.out.println(CYAN + "│" + RESET + color + BOLD + numStr + ". " + RESET
            + String.format("%-29s", label) + CYAN + "│" + RESET);
    }

    private static String centerText(String text, int width) {
        int padding = (width - text.length()) / 2;
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < padding; i++) sb.append(" ");
        sb.append(BOLD).append(text).append(RESET).append(CYAN).append(BOLD);
        while (sb.length() < width + (BOLD + RESET + CYAN + BOLD).length()) sb.append(" ");
        return sb.toString();
    }

    private static int readChoice() {
        return readIntInput();
    }

    private static int readIntInput() {
        while (true) {
            try {
                return Integer.parseInt(sc.nextLine().trim());
            } catch (NumberFormatException e) {
                System.out.print(RED + BOLD + "Invalid input. Please enter a valid number: " + RESET);
            }
        }
    }

    private static void loadingAnimation(String message) {
        System.out.print(PURPLE + BOLD + message + RESET);
        for (int i = 0; i < 5; i++) {
            System.out.print(GREEN + "." + RESET);
            sleep(200);
        }
        System.out.println(GREEN + BOLD + " Done!\n" + RESET);
    }

    private static void printActionHeader(String action) {
        System.out.println(BOLD + GREEN + "\n➤ " + action + " selected...\n" + RESET);
    }

```

```

    }

    private static void pauseForEffect() {
        System.out.print(WHITE + "\nPress ENTER to continue..." + RESET);
        sc.nextLine();
    }

    private static void exitAnimation() {
        System.out.println(YELLOW + BOLD + "\nClosing Queue Manager");
        for (int i = 0; i < 3; i++) {
            System.out.print(YELLOW + "." + RESET);
            sleep(300);
        }
        System.out.println(GREEN + BOLD + "\n✓ Goodbye!\n" + RESET);
    }

    private static void clearConsoleEffect() {
        System.out.println("\n".repeat(1));
    }

    private static void sleep(int ms) {
        try {
            Thread.sleep(ms);
        } catch (InterruptedException ignored) {
        }
    }
}

class Queue {
    private int[] tokens;
    private int front, rear, capacity, currentSize;

    public Queue(int size) {
        tokens = new int[size];
        capacity = size;
        front = 0;
        rear = -1;
        currentSize = 0;
    }

    public void enqueue(int token) {
        if (isFull()) {
            System.out.println("Queue is full. Cannot enqueue.");
        } else {
            rear = (rear + 1) % capacity;
            tokens[rear] = token;
            currentSize++;
            System.out.println("Enqueued: " + token);
        }
    }

    public int dequeue() {
        if (isEmpty()) {
            System.out.println("Queue is empty. Cannot dequeue.");
            return -1;
        } else {
            int token = tokens[front];
            front = (front + 1) % capacity;
            currentSize--;
            System.out.println("Dequeued: " + token);
            return token;
        }
    }

    public void display() {
        if (isEmpty()) {
            System.out.println("Queue is empty.");
        } else {
            System.out.print("Queue elements (front to rear): ");
            for (int i = 0; i < currentSize; i++) {

```

```

        int index = (front + i) % capacity;
        System.out.print(tokens[index] + " ");
    }
    System.out.println();
}

public boolean isFull() {
    return currentSize == capacity;
}

public boolean isEmpty() {
    return currentSize == 0;
}

public int size() {
    return currentSize;
}
}

```

Sample Output:

The screenshot shows the console output of the FIFO Queue Manager program. At the top, there is a decorative ASCII art logo. Below it, the text "FIFO QUEUE MANAGER" is displayed. The program then prints "Initializing Queue Manager..... Done!". It prompts the user to "Enter the maximum capacity of the queue: 10". Finally, it displays a "QUEUE MENU" with the following options: 1. Enqueue, 2. Dequeue, 3. Display, 4. Size, 5. isEmpty, 6. isFull, and 7. Exit.

Console output of the Queue (FIFO Queue Manager) program -- Main menu.

Question 4 – Real World Examples

For each data structure, five real-world or software examples are listed where that data structure is a natural fit, along with the reason for the fit. Advantages and disadvantages are also given for each.

1. Array:

#	Real-world / software example	Why an array fits
1	Student marks	Stores marks using indexes.
2	Book IDs	Stores books in fixed positions.
3	Monthly sales	12 months can use 12 positions.
4	Employee IDs	Stores IDs for quick access.
5	Game scores	Stores scores in order.

Advantages and Disadvantages of Array:

Advantages:

- Fast access using index.
- Easy to implement.
- Memory efficient.
- Good for fixed-size data.

Disadvantages:

- Fixed size.
- Insertion may require shifting.
- Deletion may require shifting.
- Can waste unused space.

2. Stack:

#	Real-world / software example	Why a stack fits
1	Plates	Last plate added is removed first.
2	Undo operation	Latest action is undone first.
3	Browser history	Recent page can be accessed first.
4	Function calls	Last function call finishes first.
5	Parentheses checking	Matches opening and closing brackets.

Advantages and Disadvantages of Stack:

Advantages:

- Simple LIFO structure.
- Push and pop are fast.
- Useful for undo and history.
- Easy to implement.

Disadvantages:

- Only top element is normally accessed.
- Fixed capacity with arrays.
- Can cause overflow.
- Not suitable for random access.

3. Queue:

#	Real-world / software example	Why a queue fits
1	Bank customers	First customer is served first.
2	Printer jobs	Jobs are processed in order.
3	Restaurant orders	Orders can be handled in arrival order.
4	CPU tasks	Tasks wait for processing.
5	Customer support	Requests are handled in order received.

Advantages and Disadvantages of Queue

Advantages:

- Follows FIFO.
- Simple to understand.
- Useful for scheduling.

- Enqueue/dequeue can be fast.

Disadvantages:

- Limited access.
- Fixed capacity with arrays.
- Simple implementation may require shifting.
- Not suitable for random access.

Question 5 — Advantages and Disadvantages

Based on the observations made while completing Questions 1 to 3, the advantages and disadvantages of each data structure are summarized below.

Array:

Advantages:

- Fast index-based access.
- Easy to implement.
- Suitable for fixed-size data.

Disadvantages:

- Fixed size.
- Insertion and deletion may require shifting.
- Can waste memory.

Stack:

Advantages:

- Follows LIFO.
- Push and pop are fast.
- Useful for undo and history.

Disadvantages:

- Only top element is normally accessible.
- Fixed capacity with arrays.
- Can cause overflow.

Queue:

Advantages:

- Follows FIFO.
- Useful for waiting lines and scheduling.
- Easy to implement.

Disadvantages:

- Limited access.
- Fixed capacity with arrays.

- Simple array implementation may require shifting.